

What is the recommended way to share reference data between apps?

- A.** Encapsulating the business rules in Service Actions and referencing them.
- B.** Encapsulating the business rules in Server Actions and referencing them.
- C.** Referencing Static Entities directly.
- D.** Referencing public Entities directly.

Consider a scenario where multiple apps must share a static common Menu. The Menu should have the option to switch between the Screens of these different apps. Considering the Menu is already defined in a Library, what is the best way to implement this requirement?

- A.** Implement the menu entries using external URLs with static links that point to the respective app's screens.
- B.** Create an Input Parameter of type List inside the Block containing the menu entries, and add a Link to the Block for any List element. Then, each app fetches the URL information from all the others through Service Actions and leverages the List to pass the information to the Menu.
- C.** Implement the menu entries by directly referencing the respective app's Screens.
- D.** Create one Service Action per app. Use these Service Actions to fetch the URL information from all the apps and save it in a Database Entity. Then, use the Entity records to create the Links and menu entries in the Block.

Which of the following sentences regarding Timers in ODC is correct?

- A.** A Timer can only execute following its defined schedule.
- B.** A Timer can only be created in an application.
- C.** A Timer schedule can only be defined in the ODC Portal.
- D.** A Timer can only have the same schedule across all stages.

What is the main benefit of using Libraries in ODC?

- A.** They can be strongly coupled with consumer apps for fast code execution.
- B.** They help implement business-agnostic components.
- C.** They allow sharing code with apps or other libraries.
- D.** They allow packaging code to be included in the app's container.

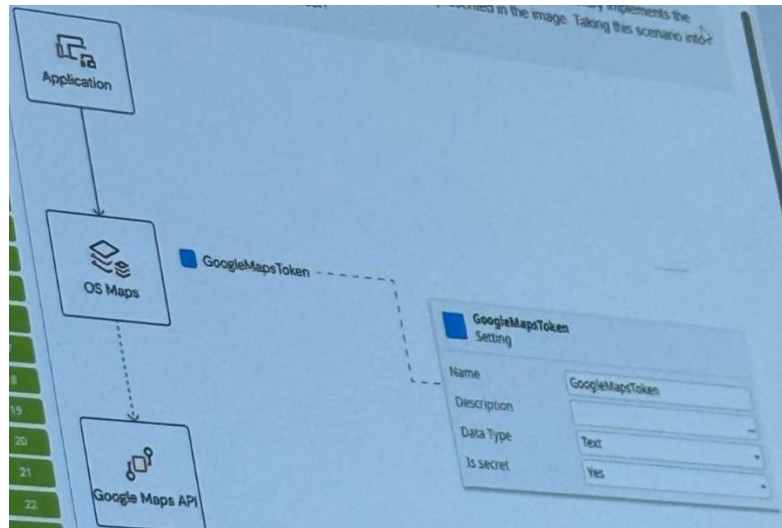
Consider that we are building two apps: Orders and Shipping. One of the requirements states that both apps should have a similar Header to be displayed on the app's Screens, containing some important business data. This business data is available and calculated in the Orders app. Which of the following steps is not part of the recommended approach to promote reusability?

- A.** Add an input parameter to the Block to receive the necessary business data.
- B.** Design the Block with the business-specific logic and data inside it.
- C.** Create a Service Action in the Orders app to fetch and calculate the required business data and reference it in the Shipping app to get the necessary data.
- D.** Create the Block in a new Library and make the apps reference it.

Considering the Architecture Design Process, which of the following is NOT a benefit of defining bounded contexts?

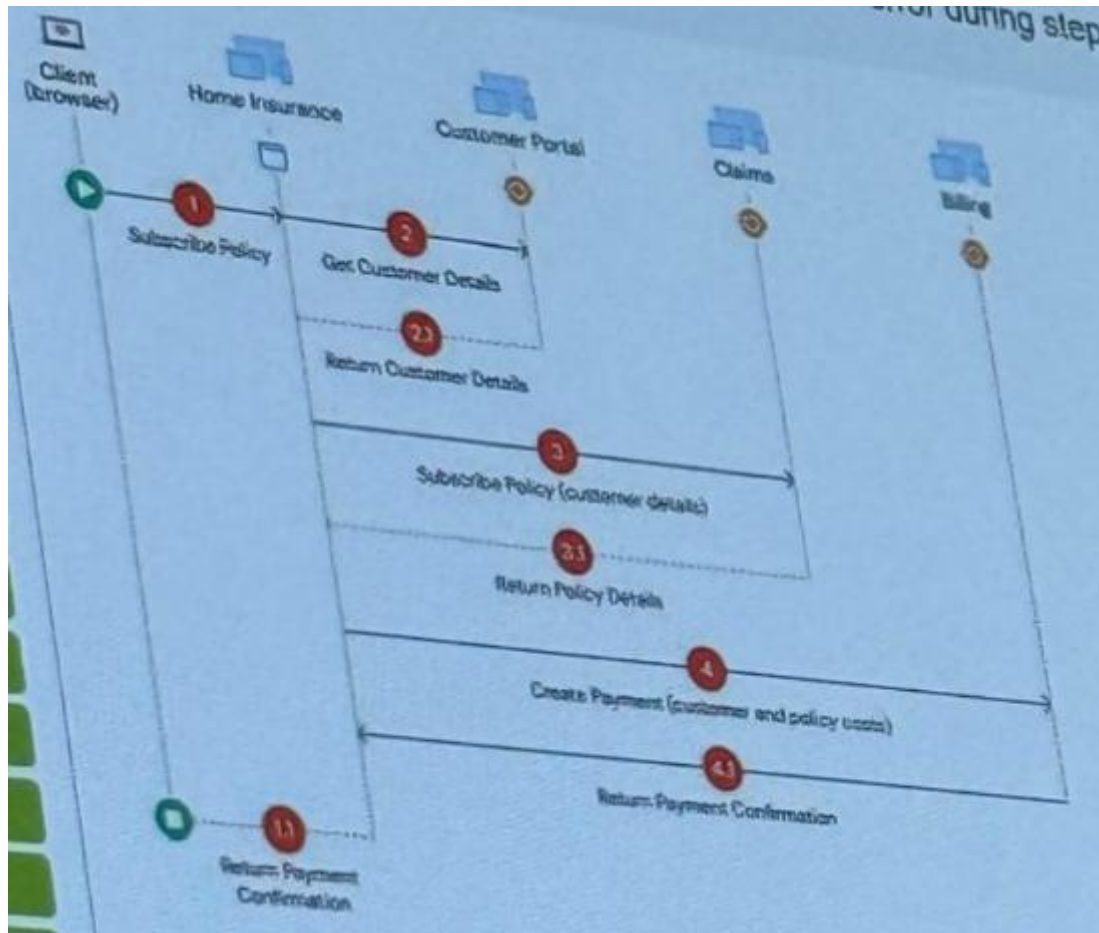
- A.** Clear definition of business experts' domain boundaries
- B.** Enhanced scalability of the system
- C.** Improved collaboration between teams
- D.** Simplified app's data model definition

Consider a scenario where multiple applications consume an ODC Library (OS Maps). This Library implements the integration with the external Google API and includes the Settings represented in the image. Taking this scenario into account, which of the following options is correct?



- A.** The Google API Token value is configurable in the Application, but you can have different values in different stages such as Development, QA, and Production.
- B.** The best practice to change the Google API Token value in a stage is to do it in ODC Studio in the Development environment, and then deploy a new app version to the next stages.
- C.** Modifying the Google API Token value is a synchronous process, meaning that when this Setting is changed in a stage, the value is immediately updated.
- D.** When the Application and OS Maps library are deployed from Development to the QA stage for the first time, the Google API Token value must be configured in the Library.

Consider the workflow below that shows the transactions between multiple Apps in an Insurance Portal. What is the best course of action in case of an operation failure or error during step 4 when creating the payment?



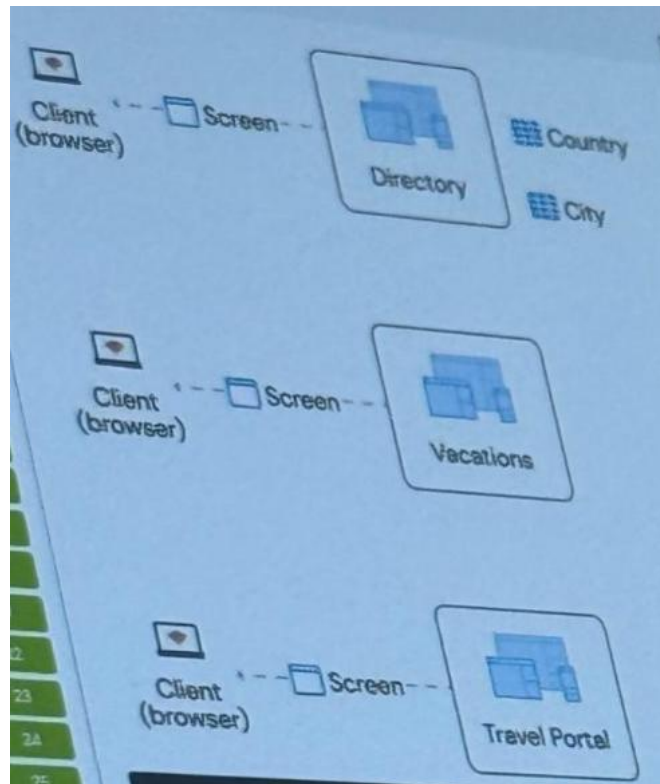
Legenda:

1. **Subscribe Policy:** O cliente inicia o pedido na app *Home Insurance*.
 1.1 Return Payment Confirmation: Resposta final ao cliente.
2. **Get Customer Details:** A app *Home Insurance* chama o *Customer Portal*.
 2.1 Return Customer Details: Resposta do *Customer Portal*.
3. **Subscribe Policy (customer details):** A app *Home Insurance* chama a app *Claims*.
 3.1 Return Policy Details: Resposta da app *Claims*.
4. **Create Payment (customer and policy costs):** A app *Home Insurance* chama a app *Billing*. **(Ponto de falha mencionado na pergunta).**
 4.1 Return Payment Confirmation: Resposta da app *Billing*.

A. Compensate the failed operation by undoing the Subscribe Policy operation.

- B. Ignore the failure and continue to the next operation.
- C. Retry the whole transaction.
- D. Delete records generated by the create payment operation.

Consider the following scenario where the Directory application has some business-agnostic data regarding Countries and Cities, which does not frequently change. Two other applications under development, Vacations and Travel Portal, will also need access to the Country and City reference data. What is the recommended approach to promote greater independence between all apps?

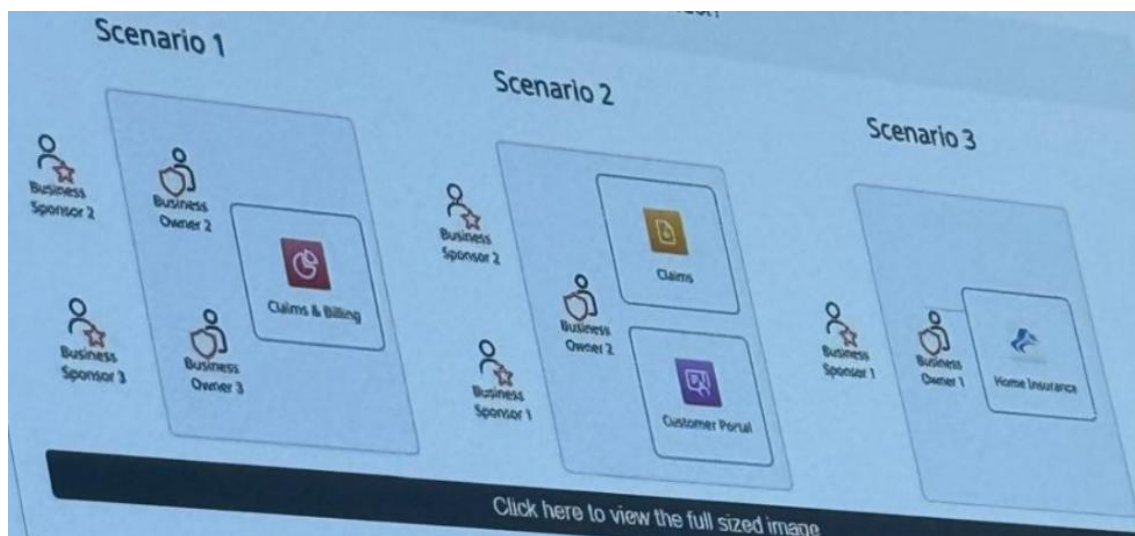


- A. Set the Country and City Entities as Public and reference them in the Vacations and Travel Portal. Use Aggregates or SQL elements to fetch data on the consumer apps when needed.
- B. Create Service Actions in the Directory app to expose the functionality required by the Vacations and Travel Portal apps, maintaining the Entities as private. Use the Service Actions in the consumer apps when needed.

C. Create a new application, Location, with the Country and City Entities. Set both Entities as Public and reference them in the Directory, Vacations, and Travel Portal. Use Aggregates or SQL elements to fetch data on the consumer apps when needed.

D. Create a new application, Location, with the Country and City Entities. Create Service Actions in the Location app to expose the functionality required by the Directory, Vacations, and Travel Portal apps, maintaining the Entities as private. Use the Service Actions in the consumer apps when needed.

Considering the scenarios below, which of the following options is correct?



A. All three scenarios follow the best practices for assembling an architecture and help promote release cycle independence.

B. Scenario 2 does not guarantee release cycle independence. This can be fixed by removing one of the Business Sponsors to avoid having multiple business sponsors for one business owner.

C. Scenario 1 does not guarantee release cycle independence. This can be fixed by splitting the Claim & Billing App into two, each App with only one Business Sponsor and one Business Owner.

D. Scenario 3 does not guarantee release cycle independence since it has a business owner and a business sponsor for one app, compromising team independence and autonomy.

Which of the following options describes a purpose for using a Cold Cache pattern in an integration with an external system?

- A.** To ensure real-time data integration with the external system.
- B.** To prevent the overload of the external system's APIs.
- C.** To reduce the development complexity required when connecting directly with external system's APIs.
- D.** To abstract several implementations of the same service in a single Library.

Considering the Architecture Design Process, which of the following options is a goal of the "Disclose" step?

- A.** Grouping business concepts into bounded contexts
- B.** Identifying dependencies between business concepts
- C.** Understanding the business requirements and concepts
- D.** Mapping business concepts to their respective teams

What happens when the number of triggered Events exceeds the consumer app's capacity for parallel execution?

- A.** The excess Events are discarded to prevent overloading the system.
- B.** The processing time for currently executing Events is automatically reduced to accommodate the new Events.
- C.** The Events will continue being added to its runtime queue until its capacity is reached.
- D.** The application pauses event processing and waits until capacity becomes available before resuming.

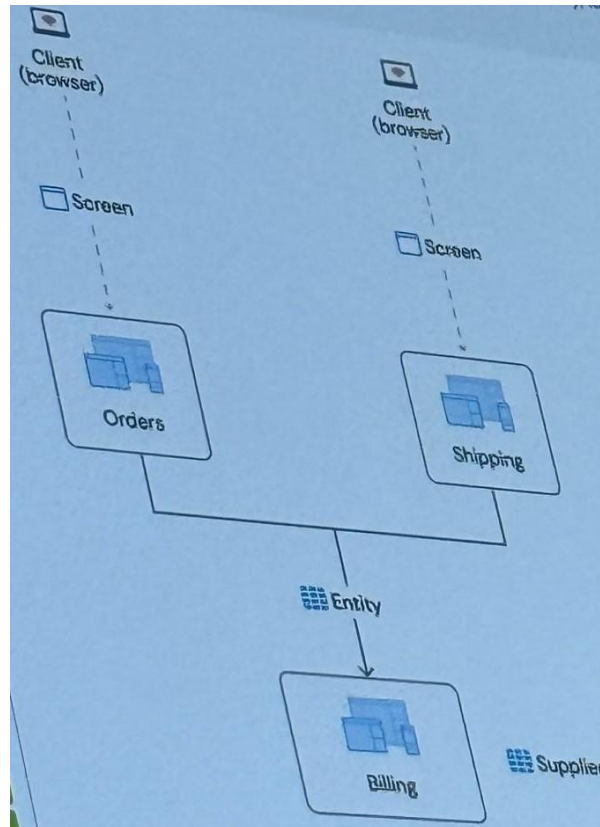
What is the main difference, if any, between an NFR and a Business Concept?

- A.** There are no differences between NFRs and Business Concepts. Both concepts describe business requirements coming from stakeholders.
- B.** NFRs focus on the technical and operational aspects of a system, while Business Concepts are more related to the broader business context.
- C.** NFRs focus on integration with external systems, while Business Concepts are more related to the broader business context.
- D.** NFRs focus on the organization helping to identify business owners, sponsors, and teams, while Business concepts are more related to the broader business context.

What is guaranteed about the order of Event delivery in OutSystems Developer Cloud?

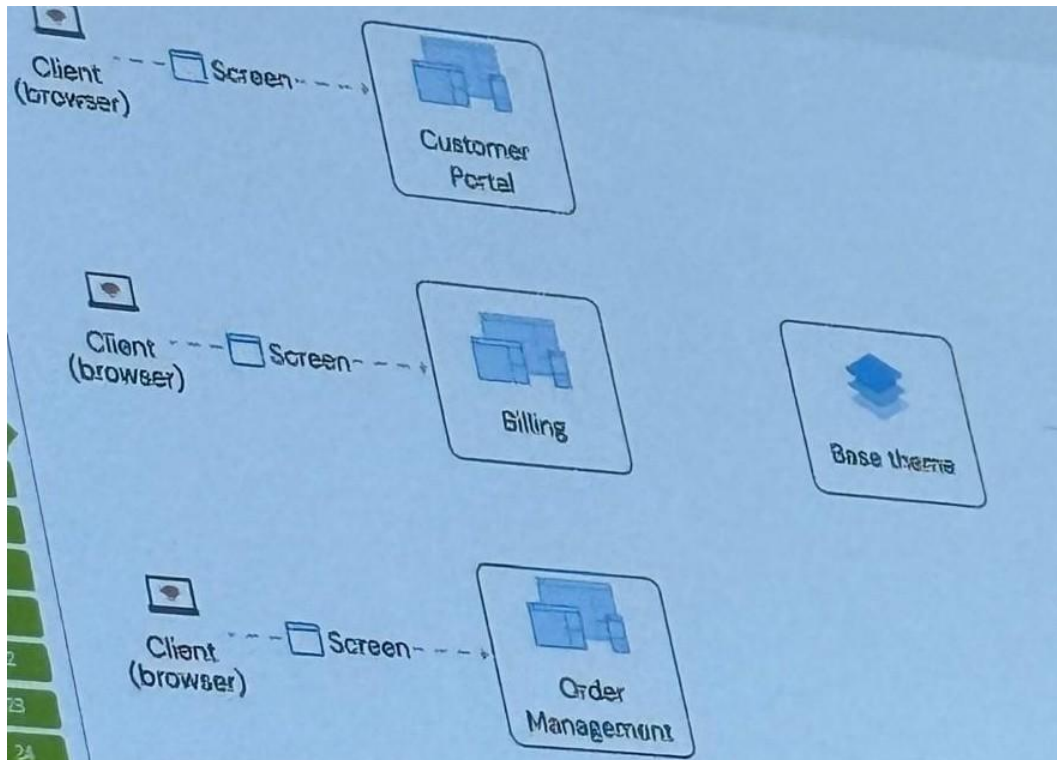
- A.** Events are delivered based on priority settings defined by the developer.
- B.** The delivery order of Events is not guaranteed.
- C.** Events are delivered strictly in a first-in, first-out (FIFO) order within the same event queue.
- D.** Events are delivered in the order they are triggered.

Consider the following scenario where the Supplier Entity is being shared with the Orders and Shipping apps. The business data in the Supplier Entity consists of thousands of records, has a low change pace, and tends to remain stable as the business evolves. The consumer apps use Aggregates to fetch the data and perform calculations with it. According to OutSystems best practices, what improvement would you make, if any, to the solution suggested in the image?



- A.** Encapsulate the business data and respective functionality in Server Actions and make the Entity private. The consumer apps can then reference and use these Server Actions.
- B.** Split the Supplier Entity information into multiple Entities, by type of Supplier, to avoid complex calculations over a long number of records.
- C.** No further improvements are needed. The solution already follows the recommended best practice.
- D.** Encapsulate the business data and respective functionality in Service Actions and make the Entity private. The consumer apps can then reference and use these Service Actions.

The scenario below shows three apps: Customer Portal, Order Management, and Billing. The Order Management and Billing are backoffice apps with the same look and feel, while the Customer Portal has a different look and feel. Despite this difference, all important styles and guidelines still come from a Base Theme. Taking this into account, what should be the approach to organize the Themes?



- A.** Create one Theme in the Customer Portal application and one new Library with the Backoffice Theme. Then, each of these new Themes references the Base Theme directly, inheriting the common CSS, and implementing the specific and required CSS. Then, the Order Management and Billing apps reference the Backoffice Theme.
- B.** Create the Portal and Backoffice Themes in the Library with the Base Theme. Then each new Theme references the Base Theme, inheriting the common CSS, and implements the specific and required CSS. The Order Management and Billing apps then reference the Backoffice Theme and the Customer Portal references the Portal Theme.
- C.** Create two new Libraries, one for the Portal Theme and another one for the Backoffice Theme. Then, each Theme references the Base Theme, inheriting the common CSS, and implements the specific and required CSS. The Order

Management and Billing apps then reference the Backoffice Theme and the Customer Portal references the Portal Theme.

D. Create two new Libraries, one for the Portal Theme and another one for the Backoffice Theme, to replace the Base Theme. Then, each Theme will have its Style Sheet with the required CSS defined, including the company branding styles. The Order Management and Billing apps then reference the Backoffice Theme and the Customer Portal references the Portal Theme.

What does it mean when we refer to “lower cognitive load” in the context of architecture assembly?

- A.** It means we are promoting a design that enables the creation of simpler code in the applications.
- B.** It means we are decreasing the mental effort required to understand and work with the Architecture Design.
- C.** It means we are adding more complexity to the architecture design.
- D.** It means we are promoting a design that will help increase the release cycle frequency.

Which of the following criteria is NOT key to choosing between a simple or distributed approach when assembling bounded concepts into apps?

- A.** Number of Product teams
- B.** Business Sponsor
- C.** Business Owner
- D.** Users expectations

Consider a scenario where you need to fix a production issue in a REST integration being consumed in a Library. At which stage should the fix be applied?

- A.** In the Production stage
- B.** In all stages simultaneously
- C.** In the Quality Assurance stage
- D.** In the Development stage

Which of the following scenarios describes a situation where there are strong references between different applications or Libraries?

- A.** All the elements from Libraries are strongly referenced by applications, except when referencing Service Actions.
- B.** Whenever an application references a Server/Client Action from another application.
- C.** Whenever an application references an element from a Library.
- D.** Whenever an application references an element from another application.

How are Libraries in ODC packaged?

- A.** They are strongly coupled with consumer apps in separate containers.
- B.** They are strongly coupled with consumer apps and included in the app's container.
- C.** They are loosely coupled with consumer apps and included in the app's container.
- D.** They are loosely coupled with consumer apps in separate containers.

Which of the following elements can an ODC Application reference / consume from an ODC Library?

- A.** Entity
- B.** Service Action
- C.** Role
- D.** Server Action

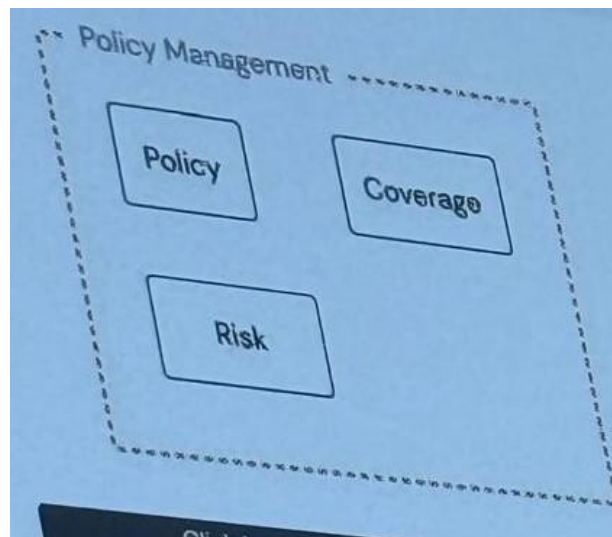
What is the purpose of the Transparency Service Pattern?

- A.** To provide different systems with the same API signatures.
- B.** To provide transparency in the routing of requests.
- C.** To provide a single service definition.
- D.** To abstract different systems into a single service.

John is working on an ODC app's Architecture Design Process. He already organized the Business Concepts after identifying bounded contexts. Which step comes next?

- A. Assemble
- B. Disclose
- C. Interviews with stakeholders
- D. Architecture blueprint design

The image below shows the Policy bounded context and its application concepts (Policy, Coverage, and Risk). Considering the naming best practices, which of the following options is incorrect?

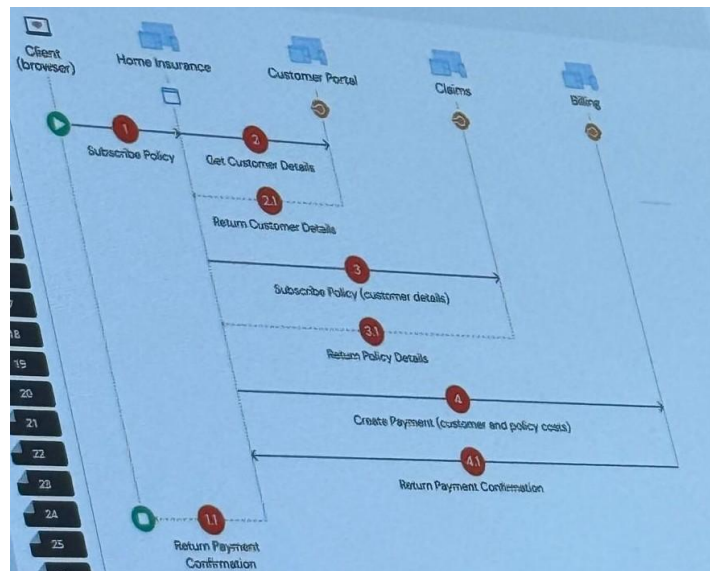


- A. Folders are a good way to organize code by grouping elements by bounded context, meaning that in this example we would have the elements, such as Actions, Screens, or Entities, all represented inside folders named *PolicyManagement*.
- B. Following a clear naming convention based on the bounded context and the app concepts, such as Policy, Risk, and Coverage, reduces the cognitive load and plays a decisive factor in improving code maintainability.
- C. The applications should be named after the bounded context they represent, meaning that the app in this example would be called *PolicyManagement*.
- D. The app's elements, such as Actions, Screens, or Entities, should be prefixed with the application concept name, followed by a clear name stating its purpose. For instance, an Action to get a risk assessment could be called *RiskGetAssessment*.

In which scenario should a Direct Integration be used instead of a Cold Cache?

- A.** When the source API does not support differential synchronization
- B.** When frequent data updates are required
- C.** When app performance is not a concern
- D.** When the app cannot tolerate data update delays

Consider a customer-facing app called Home Insurance. This app leverages important functionalities from three other apps to get customer details, subscribe to a policy, and create a payment, as represented in the image. Each interaction with any of these functionalities starts a new database transaction. What is the best approach that should be followed, if any, to ensure data consistency across all apps?



- A.** Have an Exception Handler flow in each app with the logic needed to handle and log any errors being raised by the respective operations.
- B.** Have a Timer in each app with the logic to retry the respective functionalities in case of failure. The Timer would be triggered when an error in the respective Action is raised.
- C.** There is no need to be concerned about data consistency in this simple scenario since ODC can automatically manage these sequential transactions.
- D.** The Home Insurance app needs to implement logic to orchestrate a distributed transaction, compensating failed operations in the other apps whenever errors occur.

Which of the following options accurately describes what happens when an Event is triggered in OutSystems Developer Cloud?

- A.** The Event can be handled by multiple apps, except for the app where the Event was triggered.
- B.** When the execution of the Event handler fails, the handler logic is automatically retried as many times as needed to ensure success.
- C.** The app that triggers the Event waits for it to be handled to continue its execution.
- D.** Each Event Handler that subscribes to the triggered Event executes only once.

Which of the following topics are NOT covered when disclosing the app's concepts?

- A.** Non-functional Requirements
- B.** Each app's Business Owners
- C.** Information Architecture
- D.** User Stories